

Node.js & Express Final Project

Balanced Division of Work for 5 Members — Including GitHub, README & Complete Submission Requirements

Purpose: This plan divides the backend implementation while also assigning the final documentation, presentation, screenshots, video, supporting files, GitHub workflow, integration, and ZIP packaging requirements shown in the project submission instructions.

1. Recommended Final Division

The coding responsibilities remain divided by feature, while Member 5 owns the project-level integration and submission package. All members are still responsible for testing and supporting the final integration.

Member	Main Responsibility	Core Files / Work
1	Authentication & Authorization	auth.controller.js, auth.middleware.js, role.middleware.js, auth.routes.js, auth.validation.js; register/login, bcrypt, JWT, roles/admin, rate limiting.
2	User Management & Profile	User.model.js, user.controller.js, user.routes.js, user.validation.js; user CRUD, password protection, owner/admin rules, profile/avatar handling.
3	Blog Posts & Advanced Queries	Post.model.js, post.controller.js, post.routes.js, post.validation.js, upload.middleware.js; post CRUD, cover upload, pagination, search, category/author filters, sorting, population, ownership.
4	Comments & Relations	Comment.model.js, comment.controller.js, comment.routes.js, comment.validation.js; comment create/delete, User/Post refs, population, ownership, post existence checks.
5	Infrastructure, GitHub, Documentation & Final Submission	db.js, app.js, server.js, validate.middleware.js, error.middleware.js, AppError.js, asyncWrapper.js; GitHub/repo coordination, README, presentation, screenshots, video, supporting files, final integration/testing, ZIP packaging.

2. Detailed Member Responsibilities

Member 1 — Authentication & Authorization

- Implement registration and login.
- Hash passwords using bcrypt/bcryptjs; never expose passwords in responses.
- Create JWT authentication middleware for Bearer tokens.
- Implement role-based authorization and admin checks.
- Implement authentication validation with Joi.
- Apply the required rate limiting to authentication-related routes.
- Coordinate the JWT payload/claims contract with Members 2-4.

Member 2 — User Management & Profile

- Implement the User model with name, email, password, role, avatar, and timestamps.
- Implement user CRUD routes/controllers.

- Apply owner/admin authorization rules.
- Protect password fields from being returned.
- Implement user validation with Joi.
- Handle profile/avatar updates according to the project requirements.
- Test interactions with authentication middleware.

Member 3 — Blog Posts & Advanced Queries

- Implement the Post model and its User author reference.
- Implement post CRUD and ownership checks.
- Implement cover-image upload using Multer.
- Implement public post listing with pagination.
- Implement search, category, author, and sort functionality.
- Populate author/comments where required.
- Implement post validation and coordinate the Post ↔ Comment relationship with Member 4.

Member 4 — Comments & Blog Relations

- Implement the Comment model with User and Post references.
- Implement create/delete comment endpoints.
- Validate comment input with Joi.
- Check that the referenced post exists.
- Apply comment ownership rules.
- Support population/relations needed by post responses.
- Test the full flow: authenticated user → post → comment → delete.

Member 5 — Infrastructure, GitHub, Documentation & Final Submission

- Set up MongoDB/Mongoose connection in config/db.js.
- Set up Express application and server entry points.
- Configure CORS and Helmet as required.
- Implement validation middleware for body/params/query.
- Implement centralized error middleware and AppError.
- Implement/coordinate asyncWrapper.
- Create and maintain the GitHub repository, branches, merge process, and project structure.
- Maintain the README with project name, team members, project idea, setup/run instructions, and required libraries/dependencies.
- Coordinate final integration testing across all modules.
- Prepare the Project Presentation for explaining the idea, implementation steps, results, and final outputs.
- Collect and organize Run Screenshots proving the project runs correctly and showing the final results.
- Collect and organize Project Results / Outputs such as reports, visualizations, dashboards, models, cleaned data, or other outputs applicable to the project.
- Coordinate the Short Project Video showing the project/code running successfully and highlighting important outputs/results.

- Collect any Additional Supporting Files needed during implementation and place them in the final submission package.
- Perform the final ZIP audit: source code + presentation + README + outputs + screenshots + video + supporting files.
- Ensure the ZIP filename follows the required team/project naming format.

3. Complete Submission Requirements — From the Instructor's Checklist

The final ZIP should not contain only source code. The following seven categories from the provided submission instructions must be included, as applicable to the project:

#	Required Item	What the Team Should Include	Owner / Coordinator
1	Source Code	All code files used to implement the project, including the required MVC/backend files and any project-specific files.	All members; Member 5 audits final package.
2	Project Presentation	A concise, organized presentation used during discussion to explain the project idea, implementation steps, results, and final outputs.	Member 5 coordinates; all members provide content.
3	README File	Project name, team members, project idea, how to run the project, and required libraries/dependencies (or requirements information).	Member 5
4	Project Results / Outputs	Any files produced by the project, such as reports, visualizations, dashboards, models, cleaned data, or other outputs appropriate to the project type.	All members provide; Member 5 organizes.
5	Run Screenshots	Clear screenshots demonstrating that the project/code runs correctly and showing the final results.	Member 5 coordinates; all members capture useful evidence.
6	Short Project Video	A short video showing the project/code working successfully and highlighting the important outputs/results.	Member 5 coordinates; team contributes demo content.
7	Additional Supporting Files	Any extra files used during implementation or needed by the project, organized correctly in the final ZIP.	All members; Member 5 organizes.

ZIP naming: The provided instructions show a team/project naming format example such as Team_03_Angular.zip. Use the exact team number and project name format required by your instructor.

4. Is the Work Actually Balanced?

Yes, if the team measures workload by implementation complexity and responsibility rather than by the number of files. The approximate workload is:

Member	Approx.	Why
Member 1	~20%	Security-sensitive authentication and authorization
Member 2	~18%	User CRUD plus security/ownership/profile work
Member 3	~23%	Largest feature area: posts, uploads, queries, filters, pagination
Member 4	~17%	Smaller feature area, but important relationships and ownership
Member 5	~22%	Infrastructure + GitHub + documentation + integration + all submission artifacts

Member 5 now has a substantial responsibility beyond basic setup: they own the project-level coordination and final submission package. This is why their role is intentionally broader.

5. Can Everyone Work in Parallel?

Yes. The team can work in parallel after agreeing on shared contracts before coding heavily.

Agree first on:

- Model fields and reference names.
- JWT payload/claims and authentication middleware behavior.
- Validation middleware input/output and 400-response format.
- Centralized error format and `AppError` usage.
- Common API response conventions.
- Route naming and HTTP status codes.
- How uploads and file paths are represented.

6. Suggested GitHub Workflow

Initial setup — Member 5: create the repository, add the basic project structure, `.gitignore`, README starter, and shared conventions.

Parallel branches:

- `feature/auth` — Member 1
- `feature/users` — Member 2
- `feature/posts` — Member 3
- `feature/comments` — Member 4
- `feature/infrastructure` — Member 5

Integration: merge feature branches in a controlled order, resolve conflicts, run the full API test flow, fix integration issues, then prepare the final submission artifacts.

Important: Every member should push regularly and communicate before changing shared files such as `app.js`, `server.js`, common middleware, models, or README sections.

7. Final Team Checklist Before Submission

- ☐ All required MVC/source-code files are present.
- ☐ MongoDB connection works.
- ☐ Register/login work and JWT authentication is verified.
- ☐ Role and ownership authorization rules are verified.
- ☐ Joi validation works for body/params/query where required.
- ☐ Centralized error handling works with the required status codes.
- ☐ User, post, and comment relationships/population work.
- ☐ Post pagination/search/filter/sort works.
- ☐ Upload functionality works.
- ☐ No passwords, JWT secrets, or other secrets are exposed.
- ☐ README is complete and explains setup/run instructions, team, project idea, and dependencies.

- ☐ Project Presentation is complete.
- ☐ Project Results / Outputs are collected.
- ☐ Run Screenshots clearly show successful execution and final results.
- ☐ Short Project Video is recorded and demonstrates the working project and important outputs.
- ☐ Additional Supporting Files are included when needed.
- ☐ GitHub repository is organized and final code is merged.
- ☐ Final ZIP contains all required submission categories.
- ☐ ZIP filename follows the instructor's required naming format.
- ☐ Final ZIP is opened/tested once before submission to verify nothing is missing.

Bottom line: Members 1–4 focus on feature implementation. Member 5 owns the shared infrastructure and coordinates GitHub, README, presentation, outputs, screenshots, video, supporting files, final integration, and ZIP packaging. This keeps the coding split practical while covering the complete submission checklist.